

VibeMeGood — Contextual Variance Engine

Agent Implementation Specification

Attach alongside: existing codebase (VibeMeGood-main.zip)

Read this before writing a single line of code. Several components are partially built already. Extend them — do not rebuild them.

WHAT EXISTS ALREADY — DO NOT REBUILD

These are already in the codebase. Extend, do not recreate:

Already Built	Where	What to Extend
<code>volatilityPct</code>	<code>projection/compute.ts</code>	Surface it in UI as Volatility Meter
<code>stdDev</code> , <code>pOver</code>	<code>our_projections</code> table	Use as Volatility Index display
<code>reasoning</code> object	<code>computeProjection()</code> return	Add variance signals to this object
<code>gameEnvironmentTable</code>	<code>game-environment.ts</code> schema	Add <code>blowoutProbability</code> , <code>spreadAtCapture</code>
<code>matchupHistoryTable</code>	<code>matchup-history.ts</code> schema	Already stores opponent performance data
<code>player_game_logs.homeAway</code>	Schema	Use for home/away splits immediately
<code>player_game_logs.minutes</code>	Schema	Use for minutes trend computation
<code>explain.ts</code> route	Backend routes	Add variance context to the prompt

MASTER TOGGLE ARCHITECTURE

Add to `user_settings` table schema:

```
// Add to user-settings.ts schema
varianceIntelEnabled: boolean("variance_intel_enabled").default(false),
varianceSignals: jsonb("variance_signals").default({
  fatigue: true,
  environment: true,
  usage: true,
  matchup: true,
  narrative: false,    // off by default - low signal
  referee: false,     // off by default - requires user opt-in
}),
varianceModes: jsonb("variance_modes").default({
  aggressiveWeighting: false,
  stablePicksOnly: false,
  ceilingHunterMode: false,
  excludeHighVolatility: false,
}),
experimentalLabEnabled: boolean("experimental_lab_enabled").default(false),
experimentalLabAcknowledged: boolean("experimental_lab_acknowledged").default(false)
```

Behavior:

- `varianceIntelEnabled = false` → app behaves exactly as it does today. No variance signals appear anywhere. Zero code paths changed for the default experience.
 - `varianceIntelEnabled = true` → variance scores computed, displayed, and applied per signal settings below.
 - `experimentalLabEnabled` requires a separate user confirmation dialog (see UI spec).
-

PART 1: NEW DATABASE TABLES

1A. `variance_scores` — Per-player, per-line composite scores

```
export const varianceScoresTable = pgTable("variance_scores", {
  id: serial("id").primaryKey(),
  ppLineId: integer("pp_line_id").references(() => ppLinesTable.id),
  playerId: integer("player_id").references(() => playersTable.id),
  statType: varchar("stat_type", { length: 100 }).notNull(),
})
```

```

// Composite scores (0-100)
fatigueScore: integer("fatigue_score"),          // 0 = fully rested, 100 = exha
environmentScore: integer("environment_score"), // 0 = suppressed, 100 = high-s
usageScore: integer("usage_score"),              // 0 = role shrinking, 100 = ex
matchupScore: integer("matchup_score"),          // 0 = tough matchup, 100 = exp
narrativeScore: integer("narrative_score"),      // 0 = neutral, 100 = elevated

// Derived indicators
blowoutRisk: integer("blowout_risk"),            // 0-100 probability of blowout
volatilityRating: varchar("volatility_rating", { length: 20 }), // stable|eleva
ceilingRating: integer("ceiling_rating"),        // 0-100 ceiling outcome probab
floorRating: integer("floor_rating"),            // 0-100 floor (downside) proba

// EV modifier – sum of validated signal adjustments
// Range: -0.15 to +0.15 (never exceeds 15% of projected mean)
// Only applied when varianceIntelEnabled = true
evModifier: numeric("ev_modifier").default("0"),

// Explainability – what drove the scores
signals: jsonb("signals"), // See signal structure below
warnings: jsonb("warnings"), // Array of warning tags
whyItMoves: text("why_it_moves"), // Human-readable summary

computedAt: timestamp("computed_at").defaultNow(),
}, (t) => ({
  uniq: uniqueIndex("variance_scores_unique").on(t.ppLineId),
}));

```

Signal structure stored in `signals jsonb`:

```

{
  "fatigue": {
    "isBackToBack": true,
    "daysRest": 1,
    "travelMiles": 1840,
    "timezoneShift": -2,
    "previousOT": false,
    "score": 72,
    "label": "Back-to-back, cross-country"
  },
  "environment": {
    "gameTotal": 229.5,
    "spread": -8.5,
    "blowoutProbability": 31,
    "altitude": 0,
    "homeAway": "home",

```

```

    "score": 74,
    "label": "High-total home game"
  },
  "usage": {
    "recentMinutesAvg": 36.2,
    "seasonMinutesAvg": 33.1,
    "minutesTrend": "up",
    "teammateMissed": "Ja Morant",
    "usageDelta": 8.4,
    "score": 85,
    "label": "Usage spike, key teammate out"
  },
  "matchup": {
    "opponentDefRank": 28,
    "positionalVulnerability": "weak vs centers",
    "historicalAvgVsOpponent": 28.4,
    "historicalOverRate": 0.71,
    "score": 82,
    "label": "Exploitable matchup, 71% historical over rate"
  },
  "narrative": {
    "isRevenge": false,
    "isNationalTV": true,
    "isReturnFromInjury": false,
    "contractYear": false,
    "playoffImplications": true,
    "score": 38,
    "label": "Playoff implications, national audience"
  }
}

```

Warnings array:

```
["blowout_sensitive", "back_to_back", "pace_dependent"]
```

Valid warning tags: back_to_back, three_in_four, blowout_sensitive, blowout_risk_extreme, minutes_risk, usage_volatile, rotation_unstable, boom_bust_player, pace_dependent, altitude_impact, injury_return, correlation_risk, high_volatility

1B. fatigue_data — Per-player, per-game schedule analysis

```
export const fatigueDataTable = pgTable("fatigue_data", {
```

```

id: serial("id").primaryKey(),
playerId: integer("player_id").references(() => playersTable.id),
gameId: integer("game_id").references(() => gamesTable.id),
gameDate: date("game_date").notNull(),

daysRest: integer("days_rest"),           // days since last game (0 = same day
isBackToBack: boolean("is_back_to_back").default(false),
isThreeInFour: boolean("is_three_in_four").default(false),
prevGameMinutes: numeric("prev_game_minutes"), // minutes played in last game
prevGameWasOT: boolean("prev_game_was_ot").default(false),
travelMiles: integer("travel_miles"),
timezoneShiftHours: integer("timezone_shift_hours").default(0), // negative = w
isEarlyGame: boolean("is_early_game").default(false), // 1pm or earlier local t

fatigueScore: integer("fatigue_score"), // 0 (rested) to 100 (exhausted)
computedAt: timestamp("computed_at").defaultNow(),
}, (t) => ({
  uniq: uniqueIndex("fatigue_data_unique").on(t.playerId, t.gameId),
}));

```

1C. referee_data — Ref tendency tracking

```

export const refereeDataTable = pgTable("referee_data", {
  id: serial("id").primaryKey(),
  refName: varchar("ref_name", { length: 100 }).notNull().unique(),
  sport: varchar("sport", { length: 20 }).notNull(),

  // Per-game tendencies (relative to league average = 1.0)
  foulsPerGameFactor: numeric("fouls_per_game_factor"), // >1 = foul-happy
  paceImpactFactor: numeric("pace_impact_factor"), // >1 = faster pace
  ftAttemptsFactor: numeric("ft_attempts_factor"),
  technicalFoulRate: numeric("technical_foul_rate"),

  sampleGames: integer("sample_games").default(0),
  lastUpdated: timestamp("last_updated").defaultNow(),
});

export const gameRefereeTable = pgTable("game_referee", {
  id: serial("id").primaryKey(),
  gameId: integer("game_id").references(() => gamesTable.id),
  refereeId: integer("referee_id").references(() => refereeDataTable.id),
});

```

1D. experimental_signals — Fully sandboxed

```

export const experimentalSignalsTable = pgTable("experimental_signals", {
  id: serial("id").primaryKey(),
  ppLineId: integer("pp_line_id").references(() => ppLinesTable.id),
  playerId: integer("player_id").references(() => playersTable.id),

  // All experimental — NEVER used in EV calculations
  isBirthdayGame: boolean("is_birthday_game").default(false),
  isNewShoes: boolean("is_new_shoes").default(false),           // manual flag by user
  isHaircutGame: boolean("is_haircut_game").default(false), // manual flag
  socialSpikeScore: integer("social_spike_score"),              // 0-100 if tracked
  userNotes: text("user_notes"),                                // user's own annotat

  computedAt: timestamp("computed_at").defaultNow(),
}, (t) => ({
  uniq: uniqueIndex("experimental_signals_unique").on(t.ppLineId),
}));

```

PART 2: VARIANCE SIGNAL COMPUTATION ENGINE

File: artifacts/api-server/src/lib/variance/compute-variance.ts

2A. Venue & Arena Data (hardcoded — update annually)

```

// Arena coordinates for travel distance calculation
export const ARENA_COORDS: Record<string, { lat: number; lon: number; tz: string;
  // NBA
  "LAL": { lat: 34.0430, lon: -118.2673, tz: "America/Los_Angeles", altitude: 86
  "LAC": { lat: 34.0430, lon: -118.2673, tz: "America/Los_Angeles", altitude: 86
  "GSW": { lat: 37.7680, lon: -122.3877, tz: "America/Los_Angeles", altitude: 6 }
  "PHX": { lat: 33.4457, lon: -112.0712, tz: "America/Phoenix", altitude: 108
  "SAC": { lat: 38.5805, lon: -121.4991, tz: "America/Los_Angeles", altitude: 9 }
  "POR": { lat: 45.5316, lon: -122.6668, tz: "America/Los_Angeles", altitude: 12
  "SEA": { lat: 47.6062, lon: -122.3321, tz: "America/Los_Angeles", altitude: 0 }
  "OKC": { lat: 35.4634, lon: -97.5151, tz: "America/Chicago", altitude: 366
  "DEN": { lat: 39.7487, lon: -105.0077, tz: "America/Denver", altitude: 160
  "UTA": { lat: 40.7683, lon: -111.9011, tz: "America/Denver", altitude: 128
  "MIN": { lat: 44.9795, lon: -93.2760, tz: "America/Chicago", altitude: 260
  "DAL": { lat: 32.7905, lon: -96.8103, tz: "America/Chicago", altitude: 183
  "HOU": { lat: 29.7508, lon: -95.3621, tz: "America/Chicago", altitude: 12
  "MEM": { lat: 35.1381, lon: -90.0505, tz: "America/Chicago", altitude: 84
  "NOP": { lat: 29.9490, lon: -90.0820, tz: "America/Chicago", altitude: 1 }
  "SAS": { lat: 29.4270, lon: -98.4375, tz: "America/Chicago", altitude: 198
  "CHI": { lat: 41.8807, lon: -87.6742, tz: "America/Chicago", altitude: 180

```

```

"IND": { lat: 39.7639, lon: -86.1555, tz: "America/Indiana/Indianapolis", altit
"MIL": { lat: 43.0450, lon: -87.9170, tz: "America/Chicago",          altitude: 193
"CLE": { lat: 41.4965, lon: -81.6882, tz: "America/New_York",        altitude: 199
"DET": { lat: 42.3410, lon: -83.0550, tz: "America/Detroit",        altitude: 183
"TOR": { lat: 43.6435, lon: -79.3791, tz: "America/Toronto",        altitude: 76
"BOS": { lat: 42.3662, lon: -71.0621, tz: "America/New_York",        altitude: 8 }
"NYK": { lat: 40.7505, lon: -73.9934, tz: "America/New_York",        altitude: 10
"BKN": { lat: 40.6827, lon: -73.9754, tz: "America/New_York",        altitude: 10
"PHI": { lat: 39.9012, lon: -75.1720, tz: "America/New_York",        altitude: 9 }
"WAS": { lat: 38.8981, lon: -77.0209, tz: "America/New_York",        altitude: 6 }
"ATL": { lat: 33.7573, lon: -84.3963, tz: "America/New_York",        altitude: 306
"CHA": { lat: 35.2251, lon: -80.8392, tz: "America/New_York",        altitude: 229
"ORL": { lat: 28.5392, lon: -81.3839, tz: "America/New_York",        altitude: 30
"MIA": { lat: 25.7814, lon: -80.1870, tz: "America/New_York",        altitude: 4 }
};

```

```

function haversineDistance(lat1: number, lon1: number, lat2: number, lon2: number
  const R = 3959; // miles
  const dLat = (lat2 - lat1) * Math.PI / 180;
  const dLon = (lon2 - lon1) * Math.PI / 180;
  const a = Math.sin(dLat/2)**2 + Math.cos(lat1*Math.PI/180) * Math.cos(lat2*Math
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
}

```

2B. Fatigue Score Computation

```

export function computeFatigueScore(data: {
  daysRest: number;
  isBackToBack: boolean;
  isThreeInFour: boolean;
  travelMiles: number;
  timezoneShiftHours: number;
  prevGameMinutes: number;
  prevGameWasOT: boolean;
  isEarlyGame: boolean;
}): { score: number; label: string; warnings: string[] } {
  let score = 0; // 0 = rested, 100 = exhausted
  const warnings: string[] = [];

  // Rest penalty (most important factor)
  if (data.isBackToBack) {
    score += 35;
    warnings.push("back_to_back");
  } else if (data.daysRest === 2) {
    score += 10;
  } else if (data.daysRest >= 4) {

```

```

    score -= 10; // well rested – negative fatigue
}

// Three-in-four
if (data.isThreeInFour) {
    score += 20;
    warnings.push("three_in_four");
}

// Travel penalty
if (data.travelMiles > 2000) score += 15;
else if (data.travelMiles > 1000) score += 8;
else if (data.travelMiles > 500) score += 4;

// Timezone shift (westward travel = circadian disruption for away team)
if (Math.abs(data.timezoneShiftHours) >= 3) score += 10;
else if (Math.abs(data.timezoneShiftHours) >= 2) score += 5;

// Previous game heavy load
if (data.prevGameMinutes >= 40) score += 10;
else if (data.prevGameMinutes >= 36) score += 5;

// Previous game OT
if (data.prevGameWasOT) score += 8;

// Early game disadvantage (body clock not ready)
if (data.isEarlyGame) score += 5;

score = Math.max(0, Math.min(100, score));

const label = score >= 60 ? "Heavy fatigue load"
    : score >= 40 ? "Moderate fatigue"
    : score >= 20 ? "Mild fatigue"
    : score <= -5 ? "Well rested – advantage"
    : "Normal rest";

return { score, label, warnings };
}

```

2C. Blowout Probability Computation

```

// Derives blowout risk from spread and total – already available from Odds API
export function computeBlowoutRisk(spread: number, total: number, sport: string):
    probability: number; // 0-100
    warning: string | null;
    evAdjustment: number; // -0.10 to 0 multiplier on projected counting stats

```



```

} {
  const absSpread = Math.abs(spread);

  // Blowout thresholds vary by sport
  const thresholds = {
    NBA: { moderate: 9, heavy: 14, extreme: 18 },
    NFL: { moderate: 10, heavy: 14, extreme: 17 },
    MLB: { moderate: 2.5, heavy: 3.5, extreme: 5 },
    NHL: { moderate: 1.5, heavy: 2, extreme: 2.5 },
  };

  const t = thresholds[sport as keyof typeof thresholds] ?? thresholds.NBA;

  let probability = 0;
  let evAdjustment = 0;
  let warning: string | null = null;

  if (absSpread >= t.extreme) {
    probability = 65;
    evAdjustment = -0.10; // Stars sit late, garbage time distorts
    warning = "blowout_risk_extreme"; // NO-PLAY candidate
  } else if (absSpread >= t.heavy) {
    probability = 45;
    evAdjustment = -0.06;
    warning = "blowout_sensitive";
  } else if (absSpread >= t.moderate) {
    probability = 25;
    evAdjustment = -0.02;
    warning = null;
  }

  return { probability, warning, evAdjustment };
}

```

2D. Usage Delta Computation

```

// Computes usage spike/drop from recent game logs vs season baseline
export async function computeUsageDelta(
  playerId: number,
  statType: string
): Promise<{ score: number; usageDelta: number; minutesTrend: "up" | "stable" | "
  const allLogs = await db.select().from(playerGameLogsTable)
    .where(and(eq(playerGameLogsTable.playerId, playerId), eq(playerGameLogsTable
    .orderBy(desc(playerGameLogsTable.gameDate))
    .limit(20);

```

```

if (allLogs.length < 5) return { score: 50, usageDelta: 0, minutesTrend: "stabl

const recent5 = allLogs.slice(0, 5).map(l => parseFloat(l.value.toString()));
const season = allLogs.map(l => parseFloat(l.value.toString()));

const recentAvg = recent5.reduce((a, b) => a + b, 0) / recent5.length;
const seasonAvg = season.reduce((a, b) => a + b, 0) / season.length;
const delta = ((recentAvg - seasonAvg) / seasonAvg) * 100;

let score = 50; // Neutral
let minutesTrend: "up" | "stable" | "down" = "stable";

if (delta > 15) { score = 90; minutesTrend = "up"; }
else if (delta > 8) { score = 72; minutesTrend = "up"; }
else if (delta > 3) { score = 60; minutesTrend = "up"; }
else if (delta < -15) { score = 15; minutesTrend = "down"; }
else if (delta < -8) { score = 28; minutesTrend = "down"; }
else if (delta < -3) { score = 40; minutesTrend = "down"; }

const label = delta > 8 ? `+${delta.toFixed(0)}% usage spike — expanded role`
  : delta < -8 ? `-${delta.toFixed(0)}% usage drop — reduced role`
  : "Usage stable";

return { score, usageDelta: delta, minutesTrend, label };
}

```

2E. EV Modifier — Validated Signals Only

```

// Called when varianceIntelEnabled = true
// Returns a multiplier to apply to the projected mean
// Maximum total adjustment: ±15% of projected mean
export function computeEVModifier(signals: {
  fatigueScore: number;
  blowoutAdjustment: number; // from computeBlowoutRisk
  usageDelta: number; // percentage
  aggressiveMode: boolean;
}): number {
  let modifier = 0;

  // Fatigue adjustment (validated — documented effect)
  if (signals.fatigueScore >= 60) {
    modifier -= 0.06; // -6% on back-to-back
  } else if (signals.fatigueScore >= 40) {
    modifier -= 0.03;
  } else if (signals.fatigueScore <= -5) {
    modifier += 0.02; // well-rested bonus
  }
}

```

```

}

// Blowout risk (validated - starters sit in blowouts)
modifier += signals.blowoutAdjustment; // already capped at -0.10

// Usage delta (validated - minutes → stats are correlated)
if (signals.usageDelta > 15) modifier += 0.06;
else if (signals.usageDelta > 8) modifier += 0.03;
else if (signals.usageDelta < -15) modifier -= 0.05;
else if (signals.usageDelta < -8) modifier -= 0.02;

// Aggressive mode doubles the validated signal weights
if (signals.aggressiveMode) modifier *= 2;

// Hard cap: never move projected mean more than 15%
return Math.max(-0.15, Math.min(0.15, modifier));
}

```

2F. Master Variance Compute Function

File: artifacts/api-server/src/lib/variance/index.ts

```

import { db } from "@workspace/db";
import { ppLinesTable, playersTable, gamesTable, fatigueDataTable, varianceScores } from "@workspace/db";
import { eq, and, desc } from "drizzle-orm";
import { computeFatigueScore, computeBlowoutRisk, computeUsageDelta, computeEVMod } from "@workspace/db";

export async function computeVarianceForLine(ppLineId: number): Promise<void> {
  const [row] = await db.select({ line: ppLinesTable, player: playersTable })
    .from(ppLinesTable)
    .innerJoin(playersTable, eq(ppLinesTable.playerId, playersTable.id))
    .where(eq(ppLinesTable.id, ppLineId));

  if (!row) return;
  const { line, player } = row;

  const game = line.gameId
    ? (await db.select().from(gamesTable).where(eq(gamesTable.id, line.gameId)))[0]
    : null;

  // 1. Fatigue
  const [fatigueRow] = await db.select().from(fatigueDataTable)
    .where(and(eq(fatigueDataTable.playerId, player.id), game ? eq(fatigueDataTable.gameId, game.id) : null))
    .orderBy(desc(fatigueDataTable.computedAt)).limit(1);

  const fatigueResult = fatigueRow

```

```

    ? { score: fatigueRow.fatigueScore ?? 50, label: "From schedule data", warnin
    : { score: 50, label: "No schedule data", warnings: [] }];

// 2. Environment / blowout
const [envRow] = game
  ? await db.select().from(gameEnvironmentTable).where(eq(gameEnvironmentTable.
    : [null]);

const spread = game?.spread ? Math.abs(parseFloat(game.spread.toString())) : 0;
const total = envRow?.gameTotal ? parseFloat(envRow.gameTotal.toString()) : 0;
const blowoutResult = computeBlowoutRisk(spread, total, player.sport);

// 3. Usage delta
const usageResult = await computeUsageDelta(player.id, line.statType);

// 4. Matchup
let matchupScore = 50;
let matchupLabel = "No historical matchup data";
if (game && player.teamId) {
  const oppTeamId = game.homeTeamId === player.teamId ? game.awayTeamId : game.
  if (oppTeamId) {
    const [matchup] = await db.select().from(matchupHistoryTable)
      .where(and(eq(matchupHistoryTable.playerId, player.id), eq(matchupHistory
    if (matchup && matchup.gamesPlayed && matchup.gamesPlayed >= 3) {
      const overRate = matchup.overRateAtCurrentLine ? parseFloat(matchup.overR
      matchupScore = Math.round(overRate * 100);
      matchupLabel = `${matchup.gamesPlayed} games vs this opponent - ${Math.ro
    }
  }
}

// 5. Composite warnings
const warnings = [
  ...fatigueResult.warnings,
  ...(blowoutResult.warning ? [blowoutResult.warning] : []),
  ...(usageResult.minutesTrend === "down" ? ["minutes_risk"] : []),
  ...(usageResult.usageDelta > 20 ? ["usage_volatile"] : []),
];

// 6. EV modifier (only from validated signals)
const evModifier = computeEVModifier({
  fatigueScore: fatigueResult.score,
  blowoutAdjustment: blowoutResult.evAdjustment,
  usageDelta: usageResult.usageDelta,
  aggressiveMode: false, // read from user settings in production
});

```

```

// 7. Volatility rating from existing projection data
// (stdDev and volatilityPct already computed in projection engine)

// 8. Human-readable "Why It Moves"
const reasons: string[] = [];
if (fatigueResult.score >= 50) reasons.push(fatigueResult.label);
if (blowoutResult.probability >= 30) reasons.push(`${blowoutResult.probability}%`);
if (Math.abs(usageResult.usageDelta) >= 8) reasons.push(usageResult.label);
if (matchupScore >= 70) reasons.push(matchupLabel);

const whyItMoves = reasons.length > 0 ? reasons.join(". ") : "No significant co

// Upsert to variance_scores
await db.insert(varianceScoresTable).values({
  ppLineId,
  playerId: player.id,
  statType: line.statType,
  fatigueScore: fatigueResult.score,
  environmentScore: Math.round(50 + (1 - blowoutResult.probability / 100) * 50),
  usageScore: usageResult.score,
  matchupScore,
  narrativeScore: 50, // default - narrative computation is display-only
  blowoutRisk: blowoutResult.probability,
  volatilityRating: fatigueResult.score >= 60 || blowoutResult.probability >= 4
    ? "high"
    : fatigueResult.score >= 40 || blowoutResult.probability >= 25
    ? "elevated"
    : "stable",
  evModifier: evModifier.toString(),
  signals: { fatigue: fatigueResult, environment: { blowoutRisk: blowoutResult,
  warnings,
  whyItMoves,
  computedAt: new Date(),
}).onConflictDoUpdate({
  target: [varianceScoresTable.ppLineId],
  set: {
    fatigueScore: fatigueResult.score, blowoutRisk: blowoutResult.probability,
    usageScore: usageResult.score, matchupScore, evModifier: evModifier.toStrin
    signals: { fatigue: fatigueResult, usage: usageResult, matchup: { score: ma
    warnings, whyItMoves, computedAt: new Date(),
  }
});
}

export async function computeAllVarianceScores(): Promise<number> {
  const lines = await db.select({ id: ppLinesTable.id }).from(ppLinesTable).where
  let computed = 0;

```

```
    for (const { id } of lines) {
      try { await computeVarianceForLine(id); computed++; } catch { /* non-fatal */
    }
    return computed;
  }
}
```

PART 3: BACKEND ROUTE

File: artifacts/api-server/src/routes/variance.ts

```
import { Router } from "express";
import { db } from "@workspace/db";
import { varianceScoresTable, ppLinesTable, playersTable } from "@workspace/db/sc
import { eq } from "drizzle-orm";
import { computeAllVarianceScores } from "../lib/variance";

const router = Router();

// Get variance score for a specific prop
router.get("/variance/:ppLineId", async (req, res) => {
  try {
    const [score] = await db.select().from(varianceScoresTable)
      .where(eq(varianceScoresTable.ppLineId, Number(req.params.ppLineId)));
    if (!score) return void res.json(null);
    res.json(score);
  } catch { res.status(500).json({ error: "Internal server error" }); }
});

// Get all variance scores (for slate board enrichment)
router.get("/variance", async (req, res) => {
  try {
    const scores = await db.select().from(varianceScoresTable);
    res.json(scores);
  } catch { res.status(500).json({ error: "Internal server error" }); }
});

// Trigger variance computation (called from sync/all and Force Sync)
router.post("/sync/variance", async (req, res) => {
  res.json({ status: "started" });
  try { await computeAllVarianceScores(); } catch {}
});

export default router;
```

Add to Force Sync in `sync.ts` :

```
import { computeAllVarianceScores } from "../lib/variance";
// In /sync/all, add after projections:
{ name: "variance", provider: "internal", fn: computeAllVarianceScores },
```

Add to cron in `cron.ts` :

```
// Variance scores – runs after projections at 6:30 AM
cron.schedule("30 6 * * *", () => logPull("internal", "variance-scores", computeA
```

PART 4: FRONTEND COMPONENTS

4A. Master Toggle in Settings Page

Add to Settings page, new “Variance Intelligence” section:

```
// In settings.tsx – new section

function VarianceIntelSection({ settings, onChange }: { settings: UserSettings; o
  const [showExpLab, setShowExpLab] = useState(false);

  return (
    <div className="space-y-6">
      {/* Master toggle */}
      <div className="flex items-center justify-between p-4 bg-slate-900 border b
        <div>
          <div className="font-semibold text-sm">Variance Intelligence</div>
          <div className="text-xs text-muted-foreground mt-0.5">
            Adds contextual signals to every prop: fatigue, blowout risk, usage t
            When OFF, app behaves exactly as before.
          </div>
        </div>
      </div>
      <Switch
        checked={settings.varianceIntelEnabled}
        onChange={v => onChange({ varianceIntelEnabled: v })}
      />
    </div>

    {settings.varianceIntelEnabled && (
      <div className="space-y-4 pl-4 border-l-2 border-primary/20">

        {/* Signal sub-toggles */}
```

```

<div className="text-xs font-mono uppercase tracking-wider text-muted-f
[[
  { key: "fatigue",      label: "Fatigue & Rest Modeling",    desc: "Back
  { key: "environment", label: "Game Environment",          desc: "Blow
  { key: "usage",        label: "Role & Usage Trends",        desc: "Minu
  { key: "matchup",      label: "Matchup Depth",              desc: "Hist
  { key: "narrative",    label: "Narrative Context",          desc: "Low
  { key: "referee",      label: "Referee Modeling",           desc: "Foul
].map(({ key, label, desc }) => (
  <div key={key} className="flex items-center justify-between">
    <div>
      <div className="text-sm text-foreground">{label}</div>
      <div className="text-xs text-muted-foreground">{desc}</div>
    </div>
    <Switch
      checked={settings.varianceSignals?.[key] ?? false}
      onCheckedChange={v => onChange({ varianceSignals: { ...settings.v
    />
  </div>
)}}

{/* Optimizer modes */}
<div className="text-xs font-mono uppercase tracking-wider text-muted-f
[[
  { key: "aggressiveWeighting", label: "Aggressive Variance Weighting
  { key: "stablePicksOnly",      label: "Stable Picks Mode",
  { key: "ceilingHunterMode",    label: "Ceiling Hunter Mode",
  { key: "excludeHighVolatility", label: "Exclude High Volatility",
].map(({ key, label, desc }) => (
  <div key={key} className="flex items-center justify-between">
    <div>
      <div className="text-sm text-foreground">{label}</div>
      <div className="text-xs text-muted-foreground">{desc}</div>
    </div>
    <Switch
      checked={settings.varianceModes?.[key] ?? false}
      onCheckedChange={v => onChange({ varianceModes: { ...settings.var
    />
  </div>
)}}

{/* Experimental Lab - separate gate */}
<div className="mt-6 p-4 bg-amber-950/30 border border-amber-700/40 rou
  <div className="flex items-start justify-between gap-3">
    <div>
      <div className="text-sm font-semibold text-amber-300 flex items-c
         Experimental Signals Lab

```



```

        <span className="text-[10px] font-mono bg-amber-900/50 text-amb
    </div>
    <div className="text-xs text-amber-200/60 mt-1">
        Birthday games, new shoes, haircut trends, social media activit
        Entertainment only. Never touches EV calculations.
    </div>
</div>
<Switch
    checked={settings.experimentalLabEnabled}
    onCheckedChange={v => {
        if (v && !settings.experimentalLabAcknowledged) {
            setShowExpLab(true);
        } else {
            onChange({ experimentalLabEnabled: v });
        }
    }}
/>
</div>
</div>
</div>
)}

{/* Experimental Lab confirmation dialog */}
<AlertDialog open={showExpLab} onOpenChange={setShowExpLab}>
    <AlertDialogContent className="bg-slate-900 border-slate-800">
        <AlertDialogHeader>
            <AlertDialogTitle className="font-mono text-amber-300">🚧 Experimenta
            <AlertDialogDescription className="text-slate-400 space-y-2">
                <p>The Experimental Lab contains signals with <strong className="te
                <p>These include: birthday games, new shoes, haircut patterns, soci
                <p><strong className="text-amber-300">These signals will NEVER infl
                <p>Enable this if you want to track and explore unverified patterns
            </AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
            <AlertDialogCancel>Keep Off</AlertDialogCancel>
            <AlertDialogAction
                onClick={() => onChange({ experimentalLabEnabled: true, experimenta
                className="bg-amber-800 hover:bg-amber-700 text-amber-100"
            >
                I Understand — Enable Lab
            </AlertDialogAction>
        </AlertDialogFooter>
    </AlertDialogContent>
</AlertDialog>
</div>
);

```

```
}
```

4B. Variance Badge Component (Slate Board row)

```
// components/ui/variance-badge.tsx
// Shown in Slate Board when varianceIntelEnabled = true

type VolatilityRating = "stable" | "elevated" | "high" | "boom_bust";

export function VarianceBadge({ rating, warnings }: { rating: VolatilityRating; w
  const cfg = {
    stable: { label: "STABLE",      className: "bg-emerald-900/40 text-emerald-4
    elevated: { label: "ELEVATED",  className: "bg-amber-900/40 text-amber-400 b
    high: { label: "VOLATILE",      className: "bg-rose-900/40 text-rose-400 bor
    boom_bust: { label: "BOOM/BUST", className: "bg-violet-900/40 text-violet-400
  };
  const c = cfg[rating] ?? cfg.stable;

  const warningLabels: Record<string, string> = {
    back_to_back: "B2B",
    three_in_four: "3in4",
    blowout_sensitive: "Blowout",
    blowout_risk_extreme: "▲ Blowout",
    minutes_risk: "Mins↓",
    usage_volatile: "Usage↑↑",
  };

  return (
    <Tooltip>
      <TooltipTrigger>
        <div className="flex items-center gap-1">
          <span className={`text-[9px] font-mono font-bold px-1.5 py-0.5 rounded
            {c.label}
          </span>
          {warnings.slice(0, 2).map(w => (
            <span key={w} className="text-[9px] font-mono text-slate-500 border b
              {warningLabels[w] ?? w}
            </span>
          ))}
        </div>
      </TooltipTrigger>
      <TooltipContent className="text-xs max-w-xs">
        Variance rating: {rating}. Active warnings: {warnings.join(", ") || "none
      </TooltipContent>
    </Tooltip>
  );
```

```
}
```

4C. "Why This Edge Exists" Panel (Prop Detail Sheet)

Add to `prop-detail-sheet.tsx` when `varianceIntelEnabled = true`:

```
function WhyThisEdgePanel({ variance, projection }: { variance: VarianceScore | null; projection: Projection }) {
  if (!variance) return null;

  const { signals, warnings, whyItMoves, evModifier, fatigueScore, blowoutRisk, usageScore, matchUpScore } = variance;

  const evMod = parseFloat(evModifier?.toString() ?? "0");
  const evModPct = Math.round(evMod * 100);

  return (
    <div className="space-y-4">
      {/* Header */}
      <div className="flex items-center justify-between">
        <h3 className="font-mono text-xs uppercase tracking-wider text-muted-foreground">
          {evMod !== 0 && (
            <span className={`text-xs font-mono font-bold px-2 py-0.5 rounded border ${
              evMod > 0 ? "bg-emerald-900/40 text-emerald-400 border-emerald-700/40" : "bg-rose-900/40 text-rose-400 border-rose-700/40"
            }`}>
              Model adjusted {evMod > 0 ? "+" : ""}{evModPct}% EV
            </span>
          )}
        </h3>
      </div>

      {/* Narrative summary */}
      {whyItMoves && (
        <p className="text-sm text-muted-foreground leading-relaxed border-l-2 border-l-rose-900/40 pl-2">
          {whyItMoves}
        </p>
      )}

      {/* Signal meters */}
      <div className="grid grid-cols-2 gap-3">
        {
          [
            { label: "Fatigue Load", score: fatigueScore ?? 50, invert: true, color: "rose", displayScore: 100 - (fatigueScore ?? 50) },
            { label: "Environment", score: variance.environmentScore ?? 50, invert: true, color: "emerald", displayScore: 100 - (variance.environmentScore ?? 50) },
            { label: "Usage Trend", score: usageScore ?? 50, invert: false, color: "rose", displayScore: usageScore ?? 50 },
            { label: "Matchup Edge", score: matchUpScore ?? 50, invert: false, color: "emerald", displayScore: matchUpScore ?? 50 }
          ].map(({ label, score, invert, color }) => {
            const displayScore = invert ? 100 - score : score;
            const colorMap = { rose: "bg-rose-500", emerald: "bg-emerald-500", violet: "bg-violet-500" };
            return (
              <div className="flex items-center justify-between p-2 border border-l-2 border-l-rose-900/40 border-r-2 border-r-emerald-900/40 border-bottom-2 border-bottom-violet-900/40">
                <div>{label}</div>
                <div>{displayScore}</div>
              </div>
            );
          })
        }
      </div>
    </div>
  );
}
```

```

    <div key={label} className="space-y-1.5">
      <div className="flex justify-between text-[11px] font-mono">
        <span className="text-muted-foreground">{label}</span>
        <span className="text-foreground font-bold">{displayScore}/100</s
      </div>
      <div className="h-1.5 bg-slate-800 rounded-full overflow-hidden">
        <div className={`h-full rounded-full transition-all ${colorMap[co
        </div>
      </div>
    );
  }}}
</div>

{/* Warning tags */}
{warnings && warnings.length > 0 && (
  <div className="flex flex-wrap gap-1.5">
    {(warnings as string[]).map(w => (
      <span key={w} className="text-[10px] font-mono bg-slate-800 text-ambe
        Δ {w.replace(/_/g, " ")}
      </span>
    ))}
  </div>
)}

{/* Projection reasoning (already in system) */}
{projection?.reasoning && (
  <div className="space-y-1 pt-2 border-t border-slate-800">
    <div className="text-[10px] font-mono text-muted-foreground uppercase t
    {Object.entries(projection.reasoning as Record<string, string>).filter(
      <div key={k} className="text-xs text-muted-foreground">
        <span className="text-slate-500">{k.replace(/([A-Z])/g, " $1").toLo
        <span>{v}</span>
      </div>
    ))}
  </div>
)}
</div>
);
}

```

4D. Variance Intel Sidebar Pages

Add "Variance Intel" group to `app-sidebar.tsx`:

```

// In app-sidebar.tsx, add new group:
const VARIANCE_ITEMS = [

```

```

    { title: "Stability Radar",    url: "/variance/stability",    icon: Shield },
    { title: "Fatigue Tracker",    url: "/variance/fatigue",    icon: Battery },
    { title: "Environment Board",  url: "/variance/environment", icon: Wind },
    { title: "Usage Signals",      url: "/variance/usage",      icon: Activity },
    { title: "Experimental Lab",   url: "/variance/lab",       icon: FlaskConica
];

// Only show when varianceIntelEnabled = true
{settings?.varianceIntelEnabled && (
  <SidebarGroup>
    <SidebarGroupLabel className="text-xs uppercase font-mono text-muted-foreground">
      Variance Intel
    </SidebarGroupLabel>
    <SidebarGroupContent>
      <SidebarMenu>
        {VARIANCE_ITEMS.filter(i => i.url !== "/variance/lab" || settings.experim
        <SidebarMenuItem key={item.url}>
          <SidebarMenuButton asChild isActive={location === item.url}>
            <Link href={item.url}>
              <item.icon className="w-4 h-4" />
              <span>{item.title}</span>
              {item.url === "/variance/lab" && (
                <span className="ml-auto text-[9px] font-mono bg-amber-900/40 t
              )}
            </Link>
          </SidebarMenuButton>
        </SidebarMenuItem>
      )}
    </SidebarMenu>
  </SidebarGroupContent>
</SidebarGroup>
)}

```

4E. Experimental Lab Page

File: artifacts/prizepicks/src/pages/variance/lab.tsx

```

export default function ExperimentalLab() {
  return (
    <div className="space-y-6">
      {/* Prominent disclaimer */}
      <div className="p-4 bg-amber-950/40 border border-amber-700/50 rounded-lg">
        <div className="flex items-start gap-3">
          <FlaskConical className="w-5 h-5 text-amber-400 shrink-0 mt-0.5" />
          <div>
            <div className="font-semibold text-amber-300 text-sm">Experimental Si

```

```

    <div className="text-xs text-amber-200/70 mt-1">
      Everything in this lab is exploratory and unverified. These signals
      and curiosity only. They do not influence EV calculations, optimize
      or probability scores. Use them to track patterns you observe – not
    </div>
  </div>
</div>

  /* Experimental signal cards */
  /* Birthday game tracker, new shoes log, user annotation fields */
  /* All clearly labeled EXPERIMENTAL at every display point */
</div>
);
}

```

PART 5: OPTIMIZER MODES — VARIANCE-AWARE

In the optimizer route (`routes/optimizer.ts` or client-side), add these modes when

`varianceIntelEnabled = true`:

```

const VARIANCE_OPTIMIZER_MODES = {
  "stable_grind": {
    label: "Stable Grind",
    desc: "Avoids high-volatility props. Prioritizes rested players, low blowout
    filter: (prop: any) => (prop.variance?.volatilityRating ?? "stable") !== "hig
      !(prop.variance?.warnings ?? []).includes("back_to_bac
      (prop.variance?.blowoutRisk ?? 0) < 35,
  },
  "ceiling_hunter": {
    label: "Ceiling Hunter",
    desc: "Targets props with high upside – usage spikes, elevated environment, s
    scoreBoost: (prop: any) => (prop.variance?.usageScore ?? 50) > 70 ? 0.3 : 0,
  },
  "sharp_market_only": {
    label: "Sharp Market Only",
    desc: "Standard market comparison mode. Variance signals displayed but ignore
    filter: () => true, // No variance filtering
  },
};

```

PART 6: INTEGRATION CHECKLIST

What to add to existing routes:

/api/market-intel — Add variance data to each prop row:

```
// In market-intel.ts, after fetching ext lines, also fetch variance score
const [varScore] = await db.select().from(varianceScoresTable)
  .where(eq(varianceScoresTable.ppLineId, row.line.id));

// Add to return object:
variance: varScore ? {
  volatilityRating: varScore.volatilityRating,
  blowoutRisk: varScore.blowoutRisk,
  fatigueScore: varScore.fatigueScore,
  usageScore: varScore.usageScore,
  matchupScore: varScore.matchupScore,
  warnings: varScore.warnings,
  evModifier: varScore.evModifier,
  whyItMoves: varScore.whyItMoves,
} : null,
```

/api/explain/prop/:id — Already uses Claude streaming. Add variance signals to the prompt:

```
// In explain.ts prompt construction, add:
const varScore = await db.select().from(varianceScoresTable).where(eq(varianceSco
// Inject into prompt: "Variance context: ${varScore[0]?.whyItMoves}"
```

Projection compute — Apply EV modifier when user settings have `varianceIntelEnabled = true`:

```
// In compute.ts computeAllProjections(), optionally apply variance EV modifier:
// Fetch user's variance settings
// If varianceIntelEnabled and variance score exists, multiply projected mean by
// Store adjusted value separately — never overwrite the base projection
```

PART 7: DATA SOURCES — WHERE EACH SIGNAL COMES FROM

--	--	--

Signal	Source	Already Available?
Back-to-back / days rest	Game schedule dates in <code>games</code> table	✅ Compute from existing data
Travel distance	Arena coords lookup (hardcoded above)	✅ Add hardcoded table
Timezone shift	Arena coords + IANA timezone	✅ Add with arena coords
Previous game minutes	<code>player_game_logs.minutes</code>	✅ Already in schema
Previous game OT	Game duration field (add to <code>games</code> table)	⚠️ Add <code>wasOT</code> boolean to <code>games</code>
Blowout probability	Spread + total from Odds API	✅ Already pulled
Game total / pace	<code>game_environment</code> table	✅ Already exists
Usage/minutes trend	<code>player_game_logs</code>	✅ Already exists
Home/away splits	<code>player_game_logs.homeAway</code>	✅ Already exists
Matchup history	<code>matchup_history</code> table	✅ Already exists
Referee data	refereeanalysis.com (one scrape)	⚠️ New data source needed
Weather/altitude	OpenWeatherMap (already integrated)	✅ Already integrated
Narrative signals	Manual flags / schedule metadata	⚠️ Manual or schedule scrape
Experimental signals	User-entered or manual flags	✅ User-entered

PART 8: MODELING RULES — ENFORCE THESE

1. `varianceIntelEnabled = false` must be a complete no-op. Zero code paths change. If user has not enabled it, the app runs identically to before.
2. EV modifier is **capped at ±15%** of the projected mean. Never exceed this regardless of signal combination.
3. Narrative signals (revenge, national TV, contract year) are **display-only**. They write to

`narrativeScore` in the signals blob but contribute **zero** to `evModifier`.

4. Experimental signals **never** write to `evModifier`, `volatilityRating`, or any field used in optimizer calculations.
 5. Every variance-influenced number must be **labeled**. The UI must show where a number came from. Never show a mystery adjustment.
 6. When `stablePicksOnly = true`, the optimizer simply adds `volatilityRating !== "high"` and `!warnings.includes("back_to_back")` to its filters. It does not change EV math.
 7. The projection engine's `pOver` is the **primary probability signal**. Variance scores are supplemental context. They can shift the displayed probability slightly when `varianceIntelEnabled = true`, but the core Bayesian model always drives the number.
-

PART 9: ACCEPTANCE TEST

Build is complete when:

1. Toggling Variance Intel OFF in Settings: Slate Board looks identical to pre-feature state. No variance badges visible. No extra columns.
2. Toggling Variance Intel ON: Each prop row shows a volatility badge (STABLE / ELEVATED / VOLATILE). Prop detail sheet shows "Why This Edge Exists" panel with signal meters.
3. Enabling Experimental Lab requires the confirmation dialog. Dismissing "Keep Off" leaves it disabled.
4. Stable Grind optimizer mode returns zero back-to-back players and zero props with `blowoutRisk > 35`.
5. A player confirmed on a back-to-back shows `back_to_back` warning tag and a negative EV modifier.
6. Blowout risk >60% on a prop shows `blowout_sensitive` warning and reduces projected EV by ~6%.
7. Force Sync includes variance computation as the final step.
8. Narrative signals (national TV, revenge) appear in the Why This Edge panel but contribute 0 to the displayed EV modifier.
9. Experimental Lab page shows the disclaimer banner at full prominence. Every signal on the page is labeled EXPERIMENTAL or UNVERIFIED.

10. Variance signals appear in the AI Analyst context when building the system prompt — Claude can reference fatigue and blowout risk in its analysis.